

SYSC 4001 Assignment 2 Report

Student 1: Shiven Ulaganathan - 101298172

Student 2: Anmol Thakkar - 101304659

Github Repo Part 2:

https://github.com/anmol-thakkar/SYSC4001_A2_P2

Github Repo Part 3:

https://github.com/Shiven-u/SYSC4001_A2_P3

Simulation test analysis:

Simulation 1: Basic Fork and Exec

This simulation demonstrates the use of `fork()` and `exec()`. The process performs a fork creating a child process which executes program1, once completed the parent process executes program2. Logs and snapshots show that the simulator runs the child first then the parent.

```
1 time: 24; current trace: FORK, 10
2 +-----+
3 | PID |program name |partition number | size | state |
4 +-----+
5 | 1 | init | 5 | 1 | running |
6 | 0 | init | 6 | 1 | waiting |
7 +-----+
8 time: 247; current trace: EXEC program1, 50
9 +-----+
10 | PID |program name |partition number | size | state |
11 +-----+
12 | 1 | program1 | 4 | 10 | running |
13 | 0 | init | 6 | 1 | waiting |
14 +-----+
15 time: 620; current trace: EXEC program2, 25
16 +-----+
17 | PID |program name |partition number | size | state |
18 +-----+
19 | 0 | program2 | 3 | 15 | running |
20 +-----+
21
```

system_status1

Simulation 2: Nested Fork within a Child Program

The process performs a fork, executes a CPU burst of 30ms then runs program3. The parent executes a syscall then waits for the child to finish before continuing. System snapshot shows that the child occupies the partition first with the parent waiting until execution is completed.

```
1 time: 29; current trace: FORK, 15
2 +-----+
3 | PID |program name |partition number | size | state |
4 +-----+
5 | 1 | init | 5 | 1 | running |
6 | 0 | init | 6 | 1 | waiting |
7 +-----+
8 time: 272; current trace: EXEC program3, 40
9 +-----+
10 | PID |program name |partition number | size | state |
11 +-----+
12 | 1 | program3 | 4 | 10 | running |
13 | 0 | init | 6 | 1 | waiting |
14 +-----+
15
```

system_status2

Simulation 3: Child Executes Program, Parent Executes CPU

The child executes program 4 while the parent performs a 20ms CPU burst. Logs show load time for program 4 and memory allocation to the child. Snapshot shows that partitions are

assigned correctly with the child running before the parent.

```
1  time: 26; current trace: FORK, 12
2  +-----+
3  | PID |program name |partition number | size |  state |
4  +-----+
5  |  1  |      init  |           5 |  1 | running |
6  |  0  |      init  |           6 |  1 | waiting |
7  +-----+
8  time: 334; current trace: EXEC program4, 60
9  +-----+
10 | PID |program name |partition number | size |  state |
11 +-----+
12 |  1  | program4  |           3 | 15 | running |
13 |  0  |      init  |           6 |  1 | waiting |
14 +-----+
15
```

system_status3

Simulation 4: CPU and EXEC in child, I/O in parent

The child executes a CPU burst and program after, the parent waits for child completion then does an END_IO instruction. Logs confirm child execution before parent starts and snapshot shows correct partitions and parent executing after child. The purpose of this simulation is to show that both CPU and I/O tasks can be handled in different processes.

```
1  time: 32; current trace: FORK, 18
2  +-----+
3  | PID |program name |partition number | size |  state |
4  +-----+
5  |  1  |      init  |           5 |  1 | running |
6  |  0  |      init  |           6 |  1 | waiting |
7  +-----+
8  time: 260; current trace: EXEC program5, 30
9  +-----+
10 | PID |program name |partition number | size |  state |
11 +-----+
12 |  1  | program5  |           4 | 10 | running |
13 |  0  |      init  |           6 |  1 | waiting |
14 +-----+
15
```

system_status4

Simulation 5: EXEC in child, CPU and SYSCALL in parent

Child executes program2 before the parent executes a CPU burst and SYSCALL. Logs confirm child execution before parent starts and snapshot shows correct partitions and

parent executing after child.

```
1  time: 34; current trace: FORK, 20
2  +-----+
3  | PID |program name |partition number | size |   state |
4  +-----+
5  |   1 |      init |           5 |   1 | running |
6  |   0 |      init |           6 |   1 | waiting |
7  +-----+
8  time: 332; current trace: EXEC program2, 50
9  +-----+
10 | PID |program name |partition number | size |   state |
11 +-----+
12 |   1 |  program2 |           3 |  15 | running |
13 |   0 |      init |           6 |   1 | waiting |
14 +-----+
15
```

system_status5

Conclusion:

Across all simulations the fork/exec simulator behaved as wanted, it is seen that the child processes always run before its parent, system snapshots show correct partitions, and all CPU bursts, SYSCALL and I/O operations were properly simulated.